

Maximum Planar Subgraph

Ferran Alonso

`ferran.alonso@estudiantat.upc.edu`

Arnau Cullell

`arnau.cullell@estudiantat.upc.edu`

Laia Ginabreda

`laia.ginabreda@estudiantat.upc.edu`

May 31, 2026

Contents

1	Introducción	3
2	Historia	3
3	Aplicaciones	4
4	Versión decisional y complejidad	4
4.1	Relación con eliminación de aristas	5
4.2	Idea de la reducción	5
5	Algoritmos de aproximación y aleatorizados	6
5.1	Aproximación trivial con un árbol generador	6
5.2	Algoritmo greedy de inserción de aristas	6
5.3	Algoritmo incremental aleatorizado	7
5.4	Algoritmos basados en cactus triangulares	8
5.5	Algoritmos de optimización local	9
5.6	Simulated annealing y otras heurísticas	10
5.7	Comparación de enfoques	12
5.8	Clasificación en APX	12
6	Conclusión	13

1 Introducción

El problema *Maximum Planar Subgraph* (MPS) es un problema clásico de teoría de grafos. Dado un grafo simple, no dirigido y conexo $G = (V, E)$, el objetivo es encontrar un subconjunto de aristas $F \subseteq E$ de tamaño máximo tal que el subgrafo $H = (V, F)$ sea planar [1].

Un grafo planar es un grafo que puede dibujarse en el plano de forma que sus aristas solo se intersecten en sus vértices. Es decir, se puede representar sin que dos aristas se crucen, excepto si comparten un extremo [2].

A lo largo del trabajo, consideramos grafos simples, no dirigidos y conexos. Sea $n = |V|$ el número de vértices y $m = |E|$ el número de aristas. Por la fórmula de Euler, todo grafo planar simple con $n \geq 3$ vértices tiene como mucho $3n - 6$ aristas. Además, si es maximal planar, tiene exactamente $3n - 6$ aristas y $2n - 4$ caras. A estos grafos maximales planares también se les puede llamar triangulaciones, porque todas sus caras son triangulares.

Una primera aproximación trivial para MPS consiste en escoger cualquier árbol generador del grafo. Si G es conexo, un árbol generador tiene exactamente $n - 1$ aristas y siempre es planar. Como cualquier subgrafo planar tiene como mucho $3n - 6$ aristas, se obtiene

$$\frac{n - 1}{3n - 6} = \frac{n - 1}{3(n - 2)} \approx \frac{1}{3}.$$

Por tanto, un árbol generador proporciona una aproximación de ratio aproximadamente $1/3$ [6].

Decidir si un grafo es planar puede hacerse en tiempo polinómico, incluso en tiempo lineal usando algoritmos de test de planaridad. Por tanto, la dificultad de MPS no está en comprobar la planaridad de una solución concreta, sino en encontrar el subconjunto máximo de aristas que mantiene dicha propiedad.

2 Historia

La historia del problema está muy relacionada con el desarrollo de la teoría sobre la complejidad computacional y sobre el diseño de circuitos.

Desde 1930 hasta 1960 se van desarrollando las bases del problema, ya que en los 30 Kuratowski ya formula su teorema sobre que un grafo planar no puede contener subgrafos K_5 ni $K_{3,3}$. Durante la Segunda Guerra Mundial el matemático Paul Turan pensó en un problema relacionado con la planaridad durante su estancia en un campo de trabajo forzado. Ahí tenía que transportar ladrillos desde unos hornos hasta unos almacenes a través de raíles, y mientras los transportaba con vagones se dio cuenta de que al atravesar una intersección de raíles los ladrillos se caían y se perdía tiempo. En ese momento pensó en cómo reducir el número de intersecciones entre los n hornos y los m almacenes, un problema directamente relacionado con la planaridad y los grafos bipartitos. [19]

En el 1974 John Hopcroft y Robert Tarjan demostraron que la planaridad de un grafo se podía decidir en tiempo lineal respecto al grafo $O(|V|)$, esto daba esperanzas a que el problema de MSP también fuera decidible en tiempo polinómico. En el 1977 P.C Liu

y R.C Geldmacher demostraron que MSP es un problema NP-Completo, y por lo tanto, que no existe un algoritmo polinómico que lo pueda resolver [3].

En los años 80 y 90 los investigadores se dedicaron a la búsqueda de algoritmos que aunque no fueran buenos para todos los casos igualmente hicieran un buen trabajo. Al final Michael Jünger y Petra Mutzel aplicaron técnicas con Programación Lineal Entera para encontrar MSP en grafos con un tamaño moderado.

3 Aplicaciones

En cuanto a las aplicaciones, se pueden encontrar varias, las más importantes són:

1. **Diseño de Circuitos Impresos:** en la creación de placas o circuitos los transistores y los componentes son los vértices, mientras que los cables son las aristas. Lo que se quiere conseguir es que los cables no se crucen, ya que sinó se pueden generar cortocircuitos, pero al mismo tiempo se quiere poner el máximo número de componentes, ya que así hay que poner menos capas a la placa y se abaratan los costes.
2. **Dibujo de Grafos y Visualización de Redes:** en la industria también se busca el hecho de poder visualizar redes o grafos de una forma sencilla, y para eso se necesita que el grafo sea planar, ya que de esta forma las personas entendemos mejor y más rápidamente que nos quiere decir el grafo.
3. **Arquitectura y Planificación de Instalaciones:** al diseñar los planos de una fábrica con diferentes departamentos donde se transportan materiales es muy importante el hecho de que los pasillos por donde se transportan no se crucen, ya que sinó se entorpecería el trabajo y bajaría la productividad.

4 Versión decisional y complejidad

La versión decisional del problema MPS puede formularse así:

Dado un grafo $G = (V, E)$ y un entero positivo K , ¿existe un subconjunto de aristas $F \subseteq E$ tal que (V, F) sea planar y $|F| \geq K$?

Esta pregunta equivale a preguntar si existe un subgrafo planar de G con al menos K aristas. Si se pudiera resolver esta versión decisional de forma eficiente para cualquier K , entonces se podría obtener el valor óptimo de MPS, por ejemplo mediante búsqueda binaria sobre K .

El problema decisional pertenece a NP, ya que si alguien nos da un conjunto de aristas F , podemos comprobar en tiempo polinómico que $|F| \geq K$ y que (V, F) es planar.

Además, esta versión decisional es NP-completa [3]. Esto implica que el problema de optimización MPS es NP-hard.

Hay un caso particular sencillo: si $K = |E|$, entonces la pregunta se reduce a comprobar si el grafo original G es planar. En este caso concreto, el problema sí se puede resolver en tiempo polinómico usando un test de planaridad.

4.1 Relación con eliminación de aristas

El problema MPS también puede verse como un problema de eliminación de aristas. Encontrar el máximo número de aristas que se pueden conservar manteniendo planaridad es equivalente a encontrar el mínimo número de aristas que hay que eliminar para que el grafo resultante sea planar.

Si $D \subseteq E$ es el conjunto de aristas eliminadas, entonces el subgrafo resultante es

$$G' = (V, E \setminus D).$$

Maximizar $|F|$ en MPS equivale a minimizar $|D|$, ya que

$$|F| = |E| - |D|.$$

Esta visión conecta MPS con los problemas de tipo *edge-deletion*. En el trabajo de Watanabe, Ae y Nakamura [4] se estudian problemas donde se eliminan aristas para conseguir que el grafo cumpla una propiedad π . Este problema se denota como $P_{ed}(\pi)$.

La propiedad de ser planar puede caracterizarse mediante grafos prohibidos. Por el teorema de Kuratowski, un grafo es planar si y solo si no contiene una subdivisión de K_5 ni de $K_{3,3}$. Estos dos grafos son las obstrucciones clásicas mínimas para la planaridad.

De forma intuitiva, el resultado de NP-hardness indica que decidir qué aristas eliminar para destruir todas las obstrucciones no planares puede ser computacionalmente difícil. Por tanto, MPS es NP-hard porque equivale a conservar el máximo número de aristas mientras se eliminan suficientes aristas para hacer desaparecer las estructuras que impiden la planaridad.

4.2 Idea de la reducción

La reducción utilizada en este tipo de resultados parte de un problema NP-completo, como *Connected Vertex Cover* en grafos de grado máximo 4. En este problema, dado un grafo G_0 y un entero k_0 , se pregunta si existe un subconjunto de como mucho k_0 vértices que cubra todas las aristas y que además induzca una estructura conectada.

A partir de G_0 , se construye otro grafo G_1 sustituyendo cada arista por un camino de longitud 3. Esta transformación permite relacionar la cobertura de vértices conectada con la existencia de un cierto árbol de Steiner, es decir, un árbol que conecta un conjunto de vértices importantes.

Después, se construye un nuevo grafo G_4 . Para ello, se toma una obstrucción de planaridad, como $K_{3,3}$, se elimina una arista y se obtiene una estructura casi planar. Estas estructuras se insertan en ciertas partes del grafo. La idea es que, si no se eliminan determinadas aristas, reaparecen obstrucciones no planares.

También se añaden estructuras llamadas “bandas”, formadas por muchos caminos paralelos entre dos vértices. Si el algoritmo solo puede eliminar k_4 aristas y una banda contiene $k_4 + 1$ caminos, entonces no puede destruir completamente la banda eliminando pocas aristas. Esto fuerza al algoritmo a eliminar aristas en las zonas que codifican la solución

del problema original.

La reducción funciona de la siguiente manera:

- Si G_0 tiene una solución válida, entonces existe una forma de eliminar como mucho k_4 aristas en G_4 para obtener un grafo planar.
- Si G_0 no tiene solución válida, entonces no se pueden eliminar suficientes aristas para destruir todas las obstrucciones de planaridad, y el grafo resultante no será planar.

Por tanto, resolver eficientemente el problema de eliminación de aristas para obtener planaridad permitiría resolver un problema NP-completo. Esto demuestra la NP-hardness de MPS.

5 Algoritmos de aproximación y aleatorizados

Como MPS es NP-hard, no se espera encontrar un algoritmo polinómico que siempre devuelva la solución óptima, salvo que $P = NP$. Por este motivo, se estudian algoritmos de aproximación y heurísticas.

Los algoritmos de aproximación intentan garantizar que la solución obtenida no sea demasiado mala respecto al óptimo. Las heurísticas, en cambio, no siempre tienen una buena garantía teórica, pero suelen funcionar bien en la práctica.

5.1 Aproximación trivial con un árbol generador

La aproximación más simple consiste en construir cualquier árbol generador del grafo. Si G es conexo, un árbol generador tiene $n - 1$ aristas. Además, todo árbol es planar.

Como cualquier subgrafo planar simple de n vértices tiene como mucho $3n - 6$ aristas, se cumple:

$$\frac{n - 1}{3n - 6} \geq \frac{1}{3}.$$

Por tanto, este algoritmo garantiza una aproximación de $1/3$.

La ventaja principal es que es muy fácil de implementar. Se puede construir un árbol generador usando BFS o DFS en tiempo lineal. La desventaja es que normalmente deja fuera muchas aristas que podrían añadirse sin romper la planaridad.

5.2 Algoritmo greedy de inserción de aristas

Un algoritmo más directo para MPS es el algoritmo greedy de inserción de aristas. La idea es empezar con un grafo vacío sobre el mismo conjunto de vértices e ir intentando añadir aristas una a una.

Cada vez que se añade una arista, se comprueba si el grafo sigue siendo planar. Si sigue siendo planar, la arista se conserva. Si deja de ser planar, se descarta.

```

 $H \leftarrow (V, \emptyset)$ 
para cada arista  $e \in E$  :
    si  $H + e$  es planar:
        añadir  $e$  a  $H$ 
devolver  $H$ 

```

Este algoritmo devuelve un subgrafo planar maximal. Esto significa que no se puede añadir ninguna arista más de G sin romper la planaridad. Sin embargo, maximal no significa máximo: puede existir otro subgrafo planar con más aristas.

Si el grafo original es conexo, el algoritmo greedy siempre conserva al menos un árbol generador, porque añadir una arista que conecta dos componentes diferentes no puede romper la planaridad. Por tanto, también mantiene una garantía de aproximación de $1/3$.

Si se usa un test de planaridad de tiempo lineal como el de Boyer-Myrvold, el coste del algoritmo puede expresarse como

$$O(nm),$$

ya que se realiza un test de planaridad por cada arista considerada [5].

5.3 Algoritmo incremental aleatorizado

Una variante natural del algoritmo anterior consiste en procesar las aristas en un orden aleatorio. El algoritmo es el mismo, pero antes de empezar se baraja el conjunto de aristas.

```

 $H \leftarrow (V, \emptyset)$ 
permutar aleatoriamente las aristas de  $E$ 
para cada arista  $e \in E$  :
    si  $H + e$  es planar:
        añadir  $e$  a  $H$ 
devolver  $H$ 

```

La aleatoriedad es importante porque el resultado puede depender mucho del orden en el que se procesan las aristas. Una arista añadida al principio puede impedir que más tarde se añadan varias aristas que habrían producido una solución final mejor.

Por este motivo, una estrategia práctica consiste en ejecutar el algoritmo varias veces con órdenes aleatorios distintos y quedarse con la mejor solución encontrada.

```

mejor_solucion  $\leftarrow \emptyset$ 
repetir  $R$  veces:
    permutar aleatoriamente las aristas
    aplicar el algoritmo greedy incremental
    si la nueva solución tiene más aristas:
        actualizar mejor_solucion
devolver mejor_solucion

```

Si R es el número de repeticiones, el coste total es aproximadamente

$$O(Rnm).$$

Esta técnica no mejora necesariamente la garantía teórica en el peor caso, pero suele mejorar el resultado en la práctica.

5.4 Algoritmos basados en cactus triangulares

Para obtener mejores ratios de aproximación se han estudiado algoritmos basados en estructuras triangulares. Una de estas estructuras es el cactus triangular. De forma general, un cactus triangular es un grafo en el que cada bloque es una arista o un triángulo. Esta restricción hace que la estructura sea planar, ya que los ciclos que aparecen están muy controlados y no se generan configuraciones complejas que puedan forzar cruces. La idea principal es conservar más aristas que en un árbol generador, pero sin perder la planaridad.

La motivación de estos algoritmos parte de la observación de que un árbol generador siempre proporciona una solución planar, pero contiene únicamente $n - 1$ aristas. En cambio, si es posible incorporar triángulos, se pueden añadir ciclos pequeños de longitud tres y aumentar el número de aristas conservadas. Así, los cactus triangulares pueden verse como una forma intermedia entre un árbol y un subgrafo planar más denso: mantienen una estructura suficientemente simple para garantizar la planaridad, pero permiten añadir más aristas que una solución puramente acíclica.

El algoritmo greedy basado en cactus triangulares construye inicialmente un subgrafo vacío y va buscando triángulos del grafo original cuyos tres vértices pertenezcan a componentes conexas distintas del subgrafo construido hasta el momento. Cuando encuentra un triángulo con esta propiedad, añade sus tres aristas. Esta operación une tres componentes y crea únicamente un ciclo triangular, por lo que no se introducen ciclos más grandes ni estructuras que puedan romper la planaridad. Cuando ya no es posible añadir más triángulos de este tipo, el algoritmo conecta las componentes restantes mediante aristas del grafo original, de forma similar a como se construiría un árbol generador. El resultado es una estructura triangular conectada y planar. Esta versión greedy obtiene una aproximación de $7/18$ para Maximum Planar Subgraph [8].

Una variante más potente consiste en no construir simplemente un cactus triangular maximal mediante decisiones greedy, sino buscar un cactus triangular máximo, es decir, una estructura triangular con el mayor número posible de triángulos compatibles. Para ello se utilizan técnicas de Graphic Matroid Parity. A grandes rasgos, el problema de seleccionar triángulos compatibles se transforma en un problema de paridad sobre un matroide gráfico, donde las restricciones del matroide garantizan que la estructura seleccionada no genere ciclos no permitidos, y las restricciones de paridad fuerzan que las aristas se seleccionen agrupadas de acuerdo con los triángulos. Con esta técnica se puede obtener una aproximación de $4/9$ para Maximum Planar Subgraph [7, 9].

La ventaja de esta segunda aproximación es que evita depender del orden en el que el algoritmo greedy encuentra los triángulos, ya que busca una estructura triangular óptima dentro de esta familia de subgrafos. Sin embargo, su principal desventaja es su complejidad. La reducción a Graphic Matroid Parity y los algoritmos necesarios para resolverla

son considerablemente más sofisticados que el greedy incremental o que la versión greedy del cactus triangular. Por esta razón, aunque la garantía teórica de $4/9$ es mejor, estos métodos suelen considerarse menos prácticos desde el punto de vista de la implementación. De hecho, trabajos posteriores destacan que no se conocen implementaciones sencillas de esta versión y que su interés es principalmente teórico [6].

5.5 Algoritmos de optimización local

Los algoritmos de optimización local para Maximum Planar Subgraph parten de una solución planar inicial y la van mejorando mediante cambios pequeños. La idea no es construir el subgrafo desde cero, sino modificar una solución ya existente sustituyendo algunas partes por estructuras más densas, siempre que el resultado siga siendo planar.

En este contexto, Călinescu y Uniyal proponen algoritmos que trabajan con estructuras planas sencillas, como triángulos, diamantes y copias de K_4 [11]. La intuición es que estas estructuras contienen más aristas que un árbol o un cactus triangular simple, pero siguen siendo planas. Por tanto, si una parte de la solución actual puede reemplazarse por una de estas estructuras sin introducir cruces, se obtiene un subgrafo planar con más aristas.

Uno de sus algoritmos, llamado MTLK4, parte de un cactus triangular máximo, es decir, de una solución formada principalmente por triángulos. Después busca copias de K_4 en el grafo original. Cuando encuentra una copia de K_4 que puede sustituir a uno o varios triángulos de la solución actual manteniendo la planaridad, realiza el cambio. Este proceso se repite hasta que no es posible encontrar más mejoras locales. Gracias a estas sustituciones, el algoritmo mejora ligeramente la aproximación clásica de $4/9$, alcanzando

$$\frac{4}{9} + \varepsilon, \text{ donde } \varepsilon = \frac{1}{747}$$

Aunque la mejora es pequeña, es importante desde el punto de vista teórico porque rompe una barrera que se había mantenido durante muchos años. Sin embargo, este algoritmo sigue dependiendo de técnicas complejas como Graphic Matroid Parity, lo que dificulta su implementación práctica [11].

Los mismos autores también presentan otro algoritmo llamado LDT, basado en triángulos y diamantes. Un diamante es el grafo que se obtiene al eliminar una arista de un K_4 por lo que tiene cuatro vértices y cinco aristas. Este algoritmo intenta añadir triángulos y diamantes de forma local para aumentar el número de aristas de la solución. A diferencia de MTLK4, no utiliza Graphic Matroid Parity, por lo que resulta más sencillo desde el punto de vista práctico. Su ratio de aproximación es $5/12$, menor que $4/9$, pero con una estructura algorítmica más simple [11].

Antes de continuar, conviene explicar brevemente qué es un matroide. Un matroide es una estructura matemática que sirve para generalizar la idea de independencia. En álgebra lineal, por ejemplo, un conjunto de vectores puede ser independiente si ninguno de ellos se puede obtener combinando los demás. En grafos ocurre algo parecido: un conjunto de aristas puede considerarse independiente si no forma ciclos, es decir, si forma un bosque.

Por tanto, un matroide permite tratar con una misma teoría situaciones distintas donde aparece una noción de independencia. En el caso de un matroide gráfico, los elementos

son las aristas de un grafo, y los conjuntos independientes son los conjuntos de aristas que no contienen ciclos. [13].

Graphic Matroid Parity es una variante más avanzada de esta idea. En lugar de escoger aristas sueltas, las aristas se agrupan en parejas o bloques, y el objetivo es seleccionar grupos compatibles sin romper las restricciones de independencia del matroide. En el contexto de Maximum Planar Subgraph, esta herramienta se utiliza para escoger estructuras triangulares compatibles entre sí. La utilidad de esta técnica es que permite construir subgrafos planares más densos que un árbol generador o que un cactus triangular obtenido de forma puramente greedy. [9, 10].

5.6 Simulated annealing y otras heurísticas

Otra técnica aleatorizada que puede aplicarse al problema Maximum Planar Subgraph es *simulated annealing*. A diferencia de los algoritmos greedy, que toman decisiones de forma irreversible, simulated annealing intenta mejorar una solución mediante una búsqueda aleatoria controlada. La idea principal es partir de una solución planar inicial y modificarla poco a poco, aceptando siempre los cambios que mejoran la solución y aceptando también, con cierta probabilidad, algunos cambios que la empeoran.

Esta posibilidad de aceptar soluciones peores es lo que permite al algoritmo escapar de óptimos locales. En MPS esto puede ocurrir fácilmente: una arista conservada en la solución actual puede impedir añadir varias aristas distintas que, en conjunto, darían un subgrafo planar más grande.

En el contexto de MPS, el estado del algoritmo puede representarse como un subconjunto de aristas $F \subseteq E$ tal que el subgrafo (V, F) es planar. La función objetivo es simplemente maximizar el número de aristas conservadas, es decir, maximizar $|F|$. Por tanto, una solución es mejor que otra si contiene más aristas y sigue siendo planar.

Una forma sencilla de definir los movimientos locales es intentar añadir o eliminar aristas. Por ejemplo, en cada iteración se puede escoger una arista aleatoria $e \in E$: si $e \notin F$, se prueba a añadirla; si el grafo $(V, F \cup \{e\})$ sigue siendo planar, se acepta el cambio. Si $e \in F$, se puede eliminar temporalmente y después intentar añadir otras aristas que antes no cabían. También pueden considerarse movimientos más complejos, como intercambiar una arista de la solución por una o varias aristas que no estaban en ella, siempre comprobando que el resultado final siga siendo planar.

El esquema general puede escribirse de la siguiente manera:

```

 $F \leftarrow$  solución planar inicial
 $T \leftarrow T_0$ 
mientras no se cumpla el criterio de parada:
     $F' \leftarrow$  vecino aleatorio de  $F$ 
    si  $F'$  es planar:
         $\Delta = |F'| - |F|$ 
        si  $\Delta \geq 0$  :
             $F \leftarrow F'$ 
        si  $\Delta < 0$  :
            aceptar  $F'$  con probabilidad  $e^{\Delta/T}$ 
     $T \leftarrow \alpha T$ 
devolver la mejor solución encontrada

```

El parámetro T se llama temperatura. Al principio, la temperatura es alta, por lo que el algoritmo acepta con más facilidad soluciones peores. Esto permite explorar regiones distintas del espacio de soluciones. A medida que avanza el algoritmo, la temperatura se reduce multiplicándola por un factor α , con $0 < \alpha < 1$. Cuando la temperatura es baja, el algoritmo se comporta de forma más conservadora y acepta casi únicamente cambios que mejoran la solución.

Para aplicar simulated annealing a MPS es importante escoger una buena solución inicial. Una opción simple es empezar con un árbol generador, ya que siempre es planar y garantiza $n - 1$ aristas. Sin embargo, en la práctica suele ser mejor usar como punto de partida la salida de un algoritmo greedy incremental, de un greedy aleatorizado o de un algoritmo basado en cactus triangulares. De esta forma, simulated annealing no empieza desde una solución muy pobre, sino desde un subgrafo planar relativamente denso que intenta mejorar.

El coste del algoritmo depende del número de iteraciones realizadas y del coste de comprobar la planaridad después de cada movimiento. Si se realizan I iteraciones y en cada una se ejecuta un test de planaridad de coste lineal, el coste total puede estimarse como

$$O(I(n + m)).$$

En implementaciones prácticas, este coste puede reducirse si no se comprueba la planaridad desde cero en todos los pasos, aunque esto requiere estructuras de datos más sofisticadas.

La principal ventaja de simulated annealing es su flexibilidad. Puede combinarse con otros algoritmos y permite mejorar soluciones obtenidas previamente. Además, al aceptar ocasionalmente movimientos malos, puede encontrar soluciones mejores que una búsqueda local puramente greedy. Su principal desventaja es que no proporciona una garantía de aproximación en el peor caso: la calidad final depende de parámetros como la temperatura inicial, el ritmo de enfriamiento, el número de iteraciones y el tipo de movimientos locales utilizados.

Por tanto, *simulated annealing* debe entenderse principalmente como una heurística experimental para MPS. No proporciona por sí solo una nueva garantía de aproximación,

por lo que no sustituye a los algoritmos con garantías teóricas, como los basados en cactus triangulares o en optimización local. Sin embargo, puede ser útil en la práctica para mejorar soluciones iniciales obtenidas por otros métodos [11].

5.7 Comparación de enfoques

Algoritmo	Idea principal	Garantía	Comentario
Árbol generador	Escoger un spanning tree	$1/3$	Muy simple, pero suele dejar fuera muchas aristas.
Greedy incremental	Añadir aristas si mantienen planaridad	$1/3$	Fácil de implementar y útil en la práctica.
Greedy aleatorizado	Procesar aristas en orden aleatorio	$1/3$	Puede mejorar con varios reinicios aleatorios.
Cactus triangular	Construir estructuras triangulares planares	$7/18$	Mejor que el árbol generador.
Matroid parity	Usar estructuras triangulares y paridad en matroides	$4/9$	Mejor garantía clásica, pero más complejo.
Optimización local	Mejorar una solución mediante intercambios locales	$4/9 + \varepsilon$	Mejor ratio conocida, con $\varepsilon = 1/747$.

Table 1: Resumen de algoritmos de aproximación y heurísticas para MPS.

5.8 Clasificación en APX

Todos los algoritmos estudiados en esta sección que tienen garantía teórica pertenecen al marco de **APX**, ya que son algoritmos polinómicos que devuelven una solución con una ratio de aproximación constante respecto al óptimo. En el caso de *Maximum Planar Subgraph*, esto incluye el árbol generador, el greedy de inserción de aristas, los algoritmos basados en cactus triangulares y los de optimización local.

La razón es que todos ellos garantizan conservar al menos una fracción fija del número de aristas de una solución óptima, independientemente del tamaño del grafo. Por ejemplo, el árbol generador y el greedy garantizan una aproximación de $1/3$, el cactus triangular greedy alcanza $7/18$, la versión basada en *Graphic Matroid Parity* obtiene $4/9$, y los algoritmos de optimización local mantienen ratios constantes como $5/12$ o $4/9 + 1/747$.

Por tanto, estos resultados muestran que *Maximum Planar Subgraph* pertenece a **APX**. Esto significa que, aunque no se espera resolverlo exactamente en tiempo polinómico por

ser NP-hard, sí existen algoritmos polinómicos que ofrecen una garantía constante de calidad.

Además, el problema es conocido como MaxSNP-hard, lo que indica que también es difícil desde el punto de vista de la aproximabilidad. En particular, no se espera que exista un esquema de aproximación polinómico arbitrariamente bueno, como un PTAS, salvo que se produzcan colapsos de complejidad como $P=NP$.

En cambio, *simulated annealing* y las versiones puramente aleatorizadas se consideran heurísticas si no se demuestra una garantía de aproximación propia, aunque en la práctica puedan mejorar las soluciones obtenidas.

6 Conclusión

El problema Maximum Planar Subgraph consiste en encontrar el mayor subconjunto de aristas de un grafo que conserva la planaridad. Aunque comprobar si un grafo es planar puede hacerse en tiempo polinómico, encontrar el máximo subgrafo planar es NP-hard.

La aproximación más simple consiste en devolver un árbol generador, obteniendo una ratio de $1/3$. El algoritmo greedy incremental también garantiza esta aproximación y suele dar mejores soluciones en la práctica. Su variante aleatorizada procesa las aristas en orden aleatorio y puede ejecutarse varias veces para mejorar el resultado.

Para obtener mejores garantías teóricas se usan estructuras más complejas, como cactus triangulares y algoritmos basados en *Graphic Matroid Parity*, que alcanzan una ratio de $4/9$. Más recientemente, los algoritmos de optimización local han conseguido mejorar ligeramente esta cota hasta $4/9 + \varepsilon$.

En la práctica, existe un compromiso entre calidad de la solución, dificultad de implementación y tiempo de ejecución. Los algoritmos greedy y aleatorizados son sencillos y útiles experimentalmente, mientras que los algoritmos con mejores garantías teóricas son más sofisticados.

References

- [1] M. Chimani, I. Hedtke y T. Wiedera. *Exact Algorithms for the Maximum Planar Subgraph Problem: New Models and Experiments*. SEA 2018, LIPIcs, vol. 103, 2018. Disponible en: <https://arxiv.org/abs/1804.07143>
- [2] Wikipedia. *Planar graph*. Disponible en: https://en.wikipedia.org/wiki/Planar_graph
- [3] M. R. Garey y D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Co., 1979.
- [4] T. Watanabe, T. Ae y A. Nakamura. *On the NP-hardness of edge-deletion and edge-contraction problems*. Discrete Applied Mathematics, 6(1), pp. 63–78, 1983. Disponible en: <https://www.sciencedirect.com/science/article/pii/0166218X83901014>
- [5] M. Chimani, I. Hedtke y T. Wiedera. *Limits of Greedy Approximation Algorithms for the Maximum Planar Subgraph Problem*. IWOCA 2016, Lecture Notes in Computer Science, vol. 9843, pp. 334–346, 2016. DOI: https://doi.org/10.1007/978-3-319-44543-4_26
- [6] M. Chimani, K. Klein y T. Wiedera. *A Note on the Practicality of Maximal Planar Subgraph Algorithms*. Graph Drawing and Network Visualization, GD 2016, pp. 357–364, 2016. Disponible en: <https://arxiv.org/abs/1608.07505>
- [7] G. Călinescu, C. G. Fernandes, U. Finkler y H. Karloff. *A Better Approximation Algorithm for Finding Planar Subgraphs*. Journal of Algorithms, vol. 27, no. 2, pp. 269–302, 1998. DOI: <https://doi.org/10.1006/jagm.1997.0920>
- [8] T. Poranen. *Two New Approximation Algorithms for the Maximum Planar Subgraph Problem*. Acta Cybernetica, vol. 18, no. 3, pp. 503–527, 2008. Disponible en: <https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/3735>
- [9] Z. Szigeti. *On the Graphic Matroid Parity Problem*. Journal of Combinatorial Theory, Series B, vol. 88, no. 2, pp. 247–260, 2003. Disponible en: <https://www.sciencedirect.com/science/article/pii/S009589560200045X>
- [10] L. Lovász y M. D. Plummer. *Matching Theory*. North-Holland, 1986. Disponible en: <https://shop.elsevier.com/books/matching-theory/plummer/978-0-444-87916-5>
- [11] G. Călinescu y S. Uniyal. *Local Optimization Algorithms for Maximum Planar Subgraph*. ESA 2024, LIPIcs, vol. 308, 2024. DOI: <https://doi.org/10.4230/LIPIcs.ESA.2024.38>
- [12] P. Chalermsook, A. Schmid y S. Uniyal. *A Tight Extremal Bound on the Lovász Cactus Number in Planar Graphs*. STACS 2019, LIPIcs, vol. 126, 2019. DOI: <https://doi.org/10.4230/LIPIcs.STACS.2019.19>
- [13] J. Oxley. *Matroid Theory*. 2nd ed., Oxford University Press, 2011. Disponible en: <https://global.oup.com/academic/product/matroid-theory-9780199603398>

- [14] Wikipedia. *Planarization: Finding the largest planar subgraph*. Disponible en: https://en.wikipedia.org/wiki/Planarization#Finding_the_largest_planar_subgraph
- [15] Wikipedia. *SNP complexity*. Disponible en: [https://en.wikipedia.org/wiki/SNP_\(complexity\)#MaxSNP](https://en.wikipedia.org/wiki/SNP_(complexity)#MaxSNP)
- [16] L. Faria, C. M. Herrera de Figueiredo, S. Gravier, C. F. X. de Mendonça y J. Stolfi. *On maximum planar induced subgraphs*. Discrete Applied Mathematics, vol. 154, no. 13, pp. 1774–1782, 2006. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0166218X06001338>
- [17] G. Călinescu y C. G. Fernandes. *Finding Large Planar Subgraphs and Large Subgraphs of a Given Genus*.
- [18] K. Morgan y G. Farr. *Approximation Algorithms for the Maximum Induced Planar and Outerplanar Subgraph Problems*. Journal of Graph Algorithms and Applications, vol. 11, no. 1, pp. 165–193, 2007.
- [19] Wikipedia *Pal Turán* Disponible en: https://ca.wikipedia.org/wiki/P%C3%A1l_Tur%C3%A1n